



Institut Teknologi Bandung

Direktorat Pendidikan Profesional Berkelanjutan

× PT Bank Rakyat Indonesia (Persero) Tbk

CHAPTER 2

Blok Bangunan Matematis Jaringan Saraf

Tensor, operasi tensor, dan penurunan berbasis gradien – dijelaskan lewat kode yang bisa dijalankan, bukan notasi.

Durasi 3 jam (2 sesi)

Pengajar Rahman Indra Kesuma, S.Kom., M.Cs.

Sumber Chollet & Watson, Deep Learning with Python 3e – bab 2

Tujuan Sesi

Setelah sesi ini, peserta dapat:

- 1 Menjalankan contoh MNIST pertama dari ujung ke ujung dan menyebut peran **layer, loss, optimizer, dan metric** pada tiap barisnya.
- 2 Membaca **rank, shape, dan dtype** sebuah tensor, dan memetakan data nyata (vektor, deret waktu, citra, video) ke bentuk tensornya.
- 3 Menjelaskan **operasi elemen-demi-elemen, broadcasting, hasil kali tensor, dan reshape** beserta arti geometrisnya.
- 4 Menerangkan **turunan, gradien, SGD mini-batch, dan aturan rantai**, lalu menunjuk letaknya di dalam graf komputasi.
- 5 Menulis ulang MNIST **dari nol** – Dense, Sequential, batch generator, dan lingkaran pelatihan – tanpa memakai `fit()`.

BUKU

[Bab 2 — teks penuh](#)

NOTEBOOK

[01_mnist_pertama.ipynb](#)

NOTEBOOK

[02_tensor_dan_operasi.ipynb](#)

NOTEBOOK

[03_gradien_dan_sgd.ipynb](#)

NOTEBOOK

[04_mnist_dari_nol.ipynb](#)

REPO

[Notebook resmi penulis](#)

Satu contoh, dibongkar sampai ke bawah

Bab 2 bergerak dalam satu lingkaran penuh: jalankan MNIST dengan `fit()`, bongkar tiap potongnya, lalu **tulis ulang seluruhnya dari nol** dan buktikan hasilnya sama.

1 · Contoh pertama

MNIST dalam 10 baris. Berjalan dulu, dimengerti belakangan.

BAG. 2.1

2 · Tensor

Rank, shape, dtype, irisan, sumbu batch, dan data dunia nyata.

BAG. 2.2

3 · Operasi tensor

Elemen-demi-elemen, broadcasting, matmul, reshape, dan geometrinya.

BAG. 2.3

4 · Mesinnya

Turunan, gradien, SGD, aturan rantai, autodiff.

BAG. 2.4-2.6

Kode yang bisa dijalankan adalah keterangan paling tepat dan paling tidak ambigu untuk sebuah operasi matematis.

Semangat bab 2 – notasi diganti implementasi

01

Sekali lihat jaringan saraf

MNIST – 'hello world' - nya deep learning.

Muat data: 60.000 latih, 10.000 uji

listing 2.1 – memuat MNIST

PYTHON

```
from keras.datasets import mnist

(train_images, train_labels), (test_images, test_labels) = mnist.load_data()

print(train_images.shape, train_images.dtype)
print(len(train_labels), train_labels[:10])
print(test_images.shape)
```

OUTPUT

```
(60000, 28, 28) uint8
60000 [5 0 4 1 9 2 1 3 1 4]
(10000, 28, 28)
```

ISTILAH

ARTINYA DI SINI

Sample	Satu titik data – satu citra 28×28 .
Class	Satu kategori – angka 0 sampai 9.
Label	Kelas yang melekat pada satu sample tertentu.

Model, kompilasi, pelatihan – sepuluh baris

listing 2.2-2.5 – MNIST ujung ke ujung

PYTHON

```
import keras
from keras import layers

model = keras.Sequential([
    layers.Dense(512, activation="relu"),
    layers.Dense(10, activation="softmax"),
])

model.compile(
    optimizer="adam",
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
)

train_images = train_images.reshape((60000, 28 * 28)).astype("float32") / 255
test_images = test_images.reshape((10000, 28 * 28)).astype("float32") / 255

model.fit(train_images, train_labels, epochs=5, batch_size=128)
```

Sebuah **layer** adalah *penyaring data*: ia menerima data dan mengeluarkan representasi yang lebih berguna. `softmax` di lapis akhir mengeluarkan **10 skor peluang yang berjumlah 1**.

Hasilnya – dan celah pertama yang harus dicurigai

OUTPUT

```
Epoch 1/5  
469/469 ---- 35 5ms/step - accuracy: 0.8747 - loss: 0.4358  
Epoch 5/5  
469/469 ---- 25 5ms/step - accuracy: 0.9890 - loss: 0.0361  
  
313/313 ---- 15 2ms/step - accuracy: 0.9780 - loss: 0.0745  
test accuracy: 0.978
```

98,9%

akurasi pada data latih

97,8%

akurasi pada data uji

Selisih ~1,1 poin itu bukan derau. Itu **overfitting**: model bekerja lebih baik pada yang pernah dilihatnya. Bab 5 membahasnya sebagai persoalan pokok.

Listing 2.6 – meramal

PYTHON

```
test_digits = test_images[0:10]  
predictions = model.predict(test_digits)  
print(predictions[0].argmax(), predictions[0].max(), test_labels[0])
```

OUTPUT

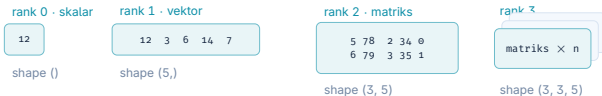
```
7 0.99993 7
```

02

Representasi data: tensor

Wadah untuk data. Tiga sifat, dan satu sumbu istimewa.

Rank, shape, dtype



MNIST sebagai tensor

```
train_images.ndim = 3 · shape (60000, 28, 28) · dtype uint8
```

sumbu ke-0 selalu sumbu sampel — dan sumbu itulah yang dipotong jadi batch

Tensor menggeneralisasi matriks ke sebarang jumlah sumbu. TensorFlow diberi nama menurut benda ini.

Jebakan istilah: **vektor 5 dimensi** $\neq$ **tensor 5 dimensi**. Yang pertama punya **satu sumbu berisi lima angka**; yang kedua punya **lima sumbu**. Salah baca di sini membuat pesan galat shape jadi tidak terbaca.

Irisan tensor dan sumbu batch

listing 2.7-2.9 – irisan

PYTHON

```
my_slice = train_images[10:100]          # 90 citra
print(my_slice.shape)

print(train_images[:, 14:, 14:].shape)    # pojok kanan-bawah 14x14
print(train_images[:, 7:-7, 7:-7].shape)  # 14x14 di tengah

batch = train_images[:128]               # batch ke-0
batch = train_images[128:256]            # batch ke-1
n = 3
batch = train_images[128 * n : 128 * (n + 1)]
```

OUTPUT

```
(90, 28, 28)
(60000, 14, 14)
(60000, 14, 14)
```

Sumbu ke-0 selalu **sumbu sampel**, dan karena model dilatih per potongan kecil, sumbu itu juga disebut **sumbu batch**. Setiap pesan galat shape yang akan Anda temui dimulai dari membaca sumbu ini.

Data dunia nyata, dan bentuk tensornya

JENIS DATA	RANK	SHAPE	CONTOH DARI BUKU
Vektor	2	<code>(samples, features)</code>	100.000 orang $\times$ (usia, jenis kelamin, penghasilan) $\rightarrow$ (100000, 3)
Deret waktu	3	<code>(samples, timesteps, features)</code>	250 hari $\times$ 390 menit $\times$ 3 nilai $\rightarrow$ (250, 390, 3)
Citra	4	<code>(samples, h, w, channels)</code>	128 citra RGB $256 \times 256 \rightarrow$ (128, 256, 256, 3)
Video	5	<code>(samples, frames, h, w, channels)</code>	4 klip $\times$ 240 bingkai $\times$ $144 \times 256 \times$ RGB $\rightarrow$ (4, 240, 144, 256, 3)

- ♦ **Channels-last** `(..., h, w, c)` – kebiasaan TensorFlow dan JAX.
- ♦ **Channels-first** `(..., c, h, w)` – kebiasaan PyTorch.
- ♦ Keras 3 memakai `image_data_format` untuk memilih; salah setelan di sini **menghasilkan galat shape yang membingungkan**.

106.168.320

nilai dalam contoh video 60 detik itu

425 MB

ukurannya pada float32

03

Gigi-giginya: operasi tensor

Semuanya bermuara pada segenggam operasi – dan semuanya punya arti geometris.

Satu lapis Dense = tiga operasi

inti sebuah lapis Dense

PYTHON

```
# output = relu(matmul(input, W) + b)
#           ^           ^           ^
#           |           |           +-- penjumlahan (dengan broadcasting)
#           |           +----- hasil kali tensor
#           +----- operasi elemen-demi-elemen

def naive_relu(x):
    assert len(x.shape) == 2
    x = x.copy()
    for i in range(x.shape[0]):
        for j in range(x.shape[1]):
            x[i, j] = max(x[i, j], 0)
    return x
```

0,02 s

NumPy tervektorisasi, 1.000 iterasi

2,45 s

gelung Python naif, 1.000 iterasi

versi tervektorisasi

PYTHON

```
z = x + y
z = np.maximum(z, 0.0)
```

Selisih **sekitar 100 kali** itu bukan soal bahasa. Versi NumPy melimpahkan pekerjaannya ke BLAS yang ditulis dalam C dan Fortran; di GPU, kode CUDA-nya tervektorisasi penuh.

Broadcasting: bentuk kecil dipaskan ke bentuk besar



tidak ada penggandaan memori sungguhan — pengulangannya hanya algoritmis

Dua langkah: tambahkan sumbu sampai rank sama, lalu ulangi sepanjang sumbu baru itu.

listing 2.11 – aturannya

PYTHON

```
X = np.random.random((64, 3, 32, 10))
y = np.random.random((32, 10))
z = np.maximum(X, y)      # y disiarkan; hasilnya (64, 3, 32, 10)
print(z.shape)
```

OUTPUT

```
(64, 3, 32, 10)
```

Hasil kali tensor dan reshape

matmul

PYTHON

```
z = np.matmul(x, y)
z = x @ y           # bentuk singkat

# aturan kecocokan:
#   x.shape[1] == y.shape[0]
# hasilnya:
#   (x.shape[0], y.shape[1])

# (a, b, c, d) @ (d,)   -> (a, b, c)
# (a, b, c, d) @ (d, e) -> (a, b, c, e)
```

reshape & transpose

PYTHON

```
x = np.array([[0., 1.],
              [2., 3.],
              [4., 5.]])      # (3, 2)

np.reshape(x, (6,))          # (6,)
np.reshape(x, (2, 3))        # (2, 3)

x = np.zeros((300, 20))
np.transpose(x).shape         # (20, 300)
```

Reshape **tidak mengubah satu pun koefisien**; ia hanya menata ulang. Itulah yang terjadi pada `train_images.reshape((60000, 784))` di contoh pertama tadi.

Tafsir geometris – dan mengapa aktivasi wajib ada

OPERASI	ARTINYA SECARA GEOMETRIS
Penjumlahan vektor	Translasi – geser objek sejauh dan searah vektor itu.
Kali matriks rotasi	Rotasi sebesar sudut θ .
Kali matriks diagonal	Penskalaan mendatar dan menegak.
Kali matriks sebarang	Transformasi linear .
Linear + translasi	Transformasi afin – persis $y = W @ x + b$.

Merangkai dua transformasi afin menghasilkan **satu** transformasi afin lagi:
 $\text{affine2}(\text{affine1}(x)) = (W_2 @ W_1) @ x + (W_2 @ b_1 + b_2)$. Jadi tumpukan Dense tanpa aktivasi **diam-diam hanyalah satu model linear**, sedalam apa pun. Fungsi aktivasi seperti ReLU-lah yang membuat ruang hipotesisnya kaya.

Gambaran yang dipakai Chollet: deep learning seperti **membuka remasan kertas**. Data yang terlipat rumit diluruskan sedikit demi sedikit oleh tiap lapis, sampai kelas-kelasnya bisa dipisah dengan bersih.

04

Mesinnya: penurunan berbasis gradien

Turunan, gradien, SGD, dan aturan rantai.

Lingkar pelatihan, dan langkah yang sulit

- 1 Ambil satu batch sampel `x` dan target `y_true`.
- 2 Jalankan model pada `x` (**lintasan maju**) $\rightarrow$ `y_pred`.
- 3 Hitung **rugi**: selisih antara `y_pred` dan `y_true`.
- 4 Perbarui bobot supaya ruginya turun sedikit.

Langkah 4 itulah yang sulit. Menyetel tiap koefisien satu per satu mustahil – jaringan modern punya **jutaan sampai miliaran parameter**. Penurunan gradien menyelesaikannya sekaligus.

Turunan adalah kemiringan hampiran linear setempat: untuk ϵ_x cukup kecil, $f(x + \epsilon_x) \approx y + a \cdot \epsilon_x$.

- `a` negatif $\rightarrow$ menaikkan `x` **menurunkan** `f(x)`.
- `a` positif $\rightarrow$ menaikkan `x` **menaikkan** `f(x)`.
- Untuk mengecilkan `f`, geser `x` **berlawanan arah tu-runannya**.

Gradien adalah turunan untuk operasi tensor. Ia tensor sebarang `W`, dan tiap koefisiennya menyatakan arah dan besar perubahan rugi bila koefisien `W` itu digeser.

$$W_1 = W_0 - \text{step} * \text{grad}(f(W_0), W_0)$$

SGD mini-batch, dan mengapa tidak diselesaikan secara analitis

- 1 Ambil batch acak (x, y_{true}) . (Kata *stochastic* datang dari **acak** ini.)
- 2 Lintasan maju $\rightarrow y_{\text{pred}}$.
- 3 Hitung rugi.
- 4 Lintasan mundur $\rightarrow$ gradien rugi terhadap parameter.
- 5 $W -= \text{learning_rate} * \text{gradient}$.

Learning rate terlalu kecil

Kekonvergenan lambat; banyak langkah untuk sedikit kemajuan.

Momentum

Perbarui berdasar gradien **sekarang dan pembaruan sebelumnya** – seperti bola menggelinding, cukup laju untuk melewati cekungan dangkal.

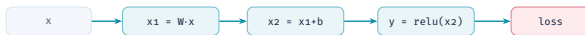
Terlalu besar

Pembaruan jadi **acak**; rugi melompat-lompat dan tidak turun.

Varian yang memperbaiki kekonvergenan disebut **optimizer**: SGD dengan momentum, Adagrad, RMSprop, Adam.

Aturan rantai di atas graf komputasi = backpropagation

lintasan maju — hitung nilai



lintasan mundur — balik arah sisi, kalikan turunan sepanjang lintasan



$$\text{grad}(\text{loss}, W) = \partial \text{loss} / \partial y \times \partial y / \partial x_2 \times \partial x_2 / \partial x_1 \times \partial x_1 / \partial W$$

aturan rantai, dijalankan mundur di atas graf — itulah backpropagation

Graf komputasi adalah graf berarah tanpa siklus. Ia membuat perhitungan bisa diperlakukan sebagai data – struktur yang terbaca mesin.

- Bila ada **beberapa lintasan** dari simpul **a** ke **b**, sumbangan tiap lintasan **dijumlahkan**.
- Kerangka kerja modern menjalankan ini sendiri: itulah **automatic differentiation**. Anda tidak pernah menulis backprop dengan tangan.

GradientTape: gradien dalam tiga baris

Listing 2.19-2.20 – autodiff

PYTHON

```
import tensorflow as tf

x = tf.Variable(3.0)
with tf.GradientTape() as tape:
    y = 2 * x + 3
print(tape.gradient(y, x))      # dy/dx = 2

# turunan kedua: pita bersarang
time = tf.Variable(0.0)
with tf.GradientTape() as outer_tape:
    with tf.GradientTape() as inner_tape:
        position = 4.9 * time ** 2
        speed = inner_tape.gradient(position, time)
    acceleration = outer_tape.gradient(speed, time)
print(acceleration)             # 9.8
```

OUTPUT

```
tf.Tensor(2.0, shape=(), dtype=float32)
tf.Tensor(9.8, shape=(), dtype=float32)
```

Contoh kedua bukan sekadar pamer: `position = 4.9 · t2` adalah jatuh bebas, dan turunan keduanya memang **percepatan gravitasi**. Autodiff mengembalikan 9,8 tanpa pernah diberi tahu rumusnya.

05

Menulis ulang dari nol

Tanpa `fit()`, tanpa `Dense` bawaan. Hasilnya harus sama.

NaiveDense dan NaiveSequential

listing 2.21-2.22 - lapis dan model

PYTHON

```
import keras
from keras import ops

class NaiveDense:
    def __init__(self, input_size, output_size, activation=None):
        self.activation = activation
        self.W = keras.Variable(shape=(input_size, output_size), initializer="uniform")
        self.b = keras.Variable(shape=(output_size,), initializer="zeros")

    def __call__(self, inputs):
        x = ops.matmul(inputs, self.W) + self.b
        return self.activation(x) if self.activation is not None else x

    @property
    def weights(self):
        return [self.W, self.b]

class NaiveSequential:
    def __init__(self, layers):
        self.layers = layers

    def __call__(self, inputs):
        x = inputs

        for layer in self.layers:
            x = layer(x)
        return x

    @property
    def weights(self):
        return [w for layer in self.layers for w in layer.weights]
```

Satu langkah pelatihan, dan gelungnya

Listing 2.24-2.26 – lingkaran pelatihan

PYTHON

```
import tensorflow as tf
from keras import optimizers

optimizer = optimizers.SGD(learning_rate=1e-3)

def one_training_step(model, images_batch, labels_batch):
    with tf.GradientTape() as tape:
        predictions = model(images_batch)
        loss = ops.sparse_categorical_crossentropy(labels_batch, predictions)
        average_loss = ops.mean(loss)
    gradients = tape.gradient(average_loss, model.weights)
    optimizer.apply_gradients(zip(gradients, model.weights))
    return average_loss

def fit(model, images, labels, epochs, batch_size=128):
    for epoch in range(epochs):
        print(f"Epoch {epoch}")
        gen = BatchGenerator(images, labels, batch_size)
        for i in range(gen.num_batches):
            images_batch, labels_batch = gen.next()
            loss = one_training_step(model, images_batch, labels_batch)
            if i % 100 == 0:
                print(f"  loss at batch {i}: {loss:.2f}")
```

Empat langkah pada slide lingkaran pelatihan tadi kini terlihat sebagai empat baris: **maju, rugi, gradien, perbarui**.

Buktinya: hasilnya memang sama

Listing 2.27 – menilai model

PYTHON

```
model = NaiveSequential([
    NaiveDense(input_size=28 * 28, output_size=512, activation=ops.relu),
    NaiveDense(input_size=512, output_size=10, activation=ops.softmax),
])
fit(model, train_images, train_labels, epochs=10, batch_size=128)

predictions = model(test_images)
predicted_labels = ops.argmax(predictions, axis=1)
matches = predicted_labels == test_labels
print(f"accuracy: {ops.mean(matches):.2f}")
```

OUTPUT

```
Epoch 0
  loss at batch 0: 6.19
  loss at batch 400: 2.21
Epoch 9
  loss at batch 0: 0.36
  loss at batch 400: 0.34
accuracy: 0.90
```

Sengaja **lebih rendah dari 97,8%**. Bedanya bukan sihir Keras: versi ini memakai SGD polos dengan learning rate $1e-3$, sedangkan yang pertama memakai **Adam**. Itu justru pelajarannya – pilihan optimizer berdampak sebesar itu.

Yang wajib terbawa dari bab 2

- 1 **Tensor** = wadah angka dengan **rank**, **shape**, **dtype**. Sumbu ke-0 adalah sumbu sampel alias sumbu batch.
- 2 **Operasi tensor** punya arti geometris. Dense tanpa aktivasi hanya transformasi afin – dan tumpukannya tetap afin.
- 3 **Belajar** = mencari nilai bobot yang mengecilkan rugi pada data latih.
- 4 **SGD mini-batch** memperbarui bobot dari gradien pada batch acak.
- 5 **Aturan rantai + autodiff** = backpropagation; graf komputasi yang mengerjakannya.
- 6 **Loss** menakar keberhasilan tugas; **optimizer** menentukan varian penurunan gradiennya.

NOTEBOOK

[01_mnist_pertama.ipynb](#)

NOTEBOOK

[04_mnist_dari_nol.ipynb](#)

BAB BERIKUT

[Bab 3 — Kerangka kerja](#)